

Attention is also Needed for Time Series*

*Prediction of Gold Price using Triformer

1st Hongda Yuan

Department of Statistics (of University of Michigan, Ann Arbor)

Ann Arbor, United States

yhongda@umich.edu

Abstract—This work focuses on implementing the Triformer model, a transformer-based architecture designed for multivariate time series forecasting. Using standard time series datasets, we analyzed the model’s ability to predict future values from high-dimensional temporal data. Our contributions include a detailed exploration of the Triformer’s components, such as triangular, variable-specific attention, along with a practical implementation pipeline. Key modifications to the original architecture include simplifying model complexity through parameter adjustments, such as reducing attention heads and dropout layers, to address overfitting. Furthermore, we developed a flexible training pipeline capable of handling diverse time series datasets. Experimental results reveal that while the Triformer achieves low training loss, challenges like overfitting and high test loss persist when applied to smaller datasets. This work provides a reproducible framework for applying state-of-the-art transformer models to real-world time series forecasting problems.

Index Terms—Multivariate Time Series, Transformer, Patch Attention

I. INTRODUCTION

Traditional statistical and machine learning approaches for time series forecasting often struggle with high-dimensional data and long-term dependencies. Transformer-based models have emerged as a powerful alternative, leveraging self-attention mechanisms to capture complex temporal patterns and dependencies over extended horizons.

The objective of this project is to thoroughly understand the structure and functionality of the Triformer [1] model, reproduce its experimental results, and evaluate its performance on new multivariate time series datasets, such as those from Kaggle.

Recent advancements in transformer-based models, including Informer [2], Autoformer, and FEDformer, have demonstrated superior performance over traditional methods and recurrent architectures by efficiently handling scalability, sparsity, and long-term dependencies. Triformer builds on these innovations by introducing additional optimizations for multivariate time series forecasting, such as dynamic weight generation and patch-based attention mechanisms.

II. METHOD

A. Problem Formulation

The task involves predicting future values of response variables in multivariate time series data, given a fixed sequence length of historical observations. Input data consists

of independent time series across multiple variables with a fixed sequence length and varying numbers of samples. The goal is to predict future values of the target variable(s) for a given prediction horizon. The datasets include publicly available multivariate time series datasets, such as the ETT datasets [4] from GitHub and custom datasets from Kaggle [3], preprocessed to handle missing values and standardize features. The model used is based on the Triformer architecture [1], which provides state-of-the-art performance in time series forecasting.

B. Methodology Overview

The Triformer model employs several novel techniques to enhance forecasting performance:

- **Patch Attention:** Temporal sequences are divided into smaller patches, enabling efficient processing and improved temporal abstraction. Attention is applied to each patch for intra-patch temporal representation learning.
- **Triangular Attention:** The attention layers are arranged in a hierarchical manner. Lower layers operate on smaller patches of the time series, while upper layers process larger patches. The number of patches in consecutive layers must successively divide the input sequence length.
- **Skip Connections:** Skip connections iteratively aggregate intermediate outputs from each attention layer, providing additional context to the prediction from all the previous layers.
- **Variable Memory Matrix:** A memory mechanism dynamically generates weights for each variable in the time series, enabling a more expressive and lightweight representation. This mechanism is implemented using factorized memory matrices and learnable memory embeddings.
- **Recurrent Relation and Gating Procedure:** Efficiency of Triformer comes with the price that the connections between patches are broken. Having recurrent relation and gating procedure restores the links between patches within each triangular attention layer.

III. RESULTS

A. Data Pipeline and Model Setup

We implemented the following data pipeline to transform raw time series data from CSV files into training samples and testing samples:

TABLE I
PARAMETERS

(Hyper) parameter	Description
sequence length	Length of the input time series sequence
horizon	Length of the output prediction window
input dimension	Number of input features (covariates)
output dimension	Number of prediction variables
channels	Number of embedding channels
patch sizes	Patch sizes for patch-wise attention
dynamic	Whether to use dynamic weight generation
batch size	Number of samples per batch
epochs	Number of training epochs
learning rate	Learning rate for the optimizer

- 1 Sorting by the time column to ensure chronological order.
- 2 Handling missing values using mean imputation and standardizing.
- 3 Using sliding windows to slice the dataframe, into input-output pairs, splitting the data for training and testing.
- 4 Encoding covariates and targets as tensors for input into the model, batching the training and testing sets using DataLoader.

B. Declaration of Parameters

We briefly define and explain the parameters we talk about and used in Triformer in Table I.

C. Training and Testing

After setting up the model, we fit the model to the training data, and test them on the test data. The loss function we used is MSE. We record models performances in Table II.

Due to space constraints, (hyper) parameters remaining same across all the trainings are omitted.

In order to demonstrate the model prediction results, we randomly selected several sets of samples from the test set and make predictions on them. The comparison between the model outputs and ground truth can be found in Figure 1.

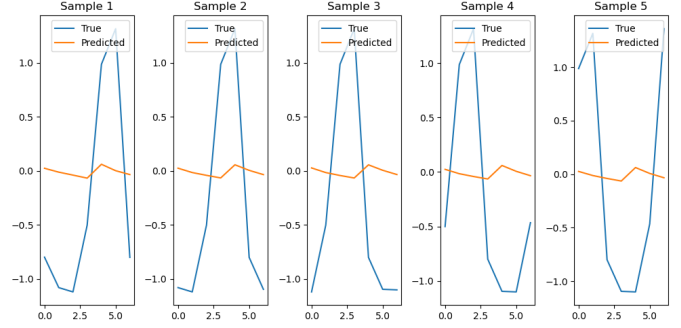
D. Interpretation

Both from the figures and the tables, we can see that the model does not predict well, for different inputs, the model outputs the same prediction, and is unable to predict drastic changes in the time series over a short period of time. During the training process we observed that the training loss kept decreasing and ended up being much lower than the testing loss. We believe this is mainly due to overfitting.

TABLE II
MODEL PERFORMANCE

Dataset	Sequence Length	Input Dimension	Patch Size	Test Loss
ETTm1	96	6	[2,4,12]	1.1206
ETTm1	30	6	[2,3,5]	1.5163
ETTh1	96	6	[2,4,12]	1.2573
ETTh1	30	6	[2,3,5]	1.1798
Kaggle	96	2	[2,4,12]	1.0778
Kaggle	30	2	[2,3,5]	1.2521
Kaggle	96	8	[2,4,12]	1.1338
Kaggle	30	8	[2,3,5]	1.039

Fig. 1. Predictions



IV. CONCLUSION

In this project, we explored the Triformer architecture for multivariate time series forecasting, reproducing its key components and adapting it to new datasets. Our work included an in-depth examination of the model's triangular attention mechanism, variable-specific memory structure, and dynamic weight generation. We integrated these features into a practical pipeline for data preprocessing, training, and evaluation using real-world datasets.

The major accomplishments include: utilized existing resource on GitHub, designed and implemented training pipeline, adapted Triformer to real-world data, utilized HPC for model training.

We analysed the reasons for the overfitting problem, and here are some of them:

- We've rewritten the training process from scratch, it doesn't detect overfitting and therefore has no early-stop options.
- According to the ablation study in the paper, the total number of parameters in the model is far greater than the samples in the current training sets, hence tuning hyperparameters may not work as well.
- There are no pre-trained models on the web, so we can't use existing model directly.

A well-documented executable codebase can be found on GitHub [5].

REFERENCES

- [1] R.-G. Cirstea, C. Guo, B. Yang, T. Kieu, X. Dong, and S. Pan, "Triformer: Triangular, variable-specific attentions for long sequence multivariate time series forecasting," *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2022.
- [2] H. Zhou, J. Li, S. Zhang, S. Zhang, M. Yan, and H. Xiong, "Expanding the prediction capacity in long sequence time-series forecasting," *Artificial Intelligence*, vol. 318, p. 103886, 2023.
- [3] S. Kumar, "2019-2024 US Stock Market Data," Kaggle, 2024, [Online]. Available: <https://www.kaggle.com/dsv/7553516>.
- [4] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *The Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI 2021)*, Virtual Conference, vol. 35, no. 12, pp. 11106–11115, 2021.
- [5] H. Yuan, "stats507-homework," GitHub repository, 2024. [Online]. Available: <https://github.com/TonyYuanMD/stats507-homework>